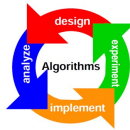


Estrategias algorítmicas

José A. Rodríguez Melquiades

7 de junio de 2026

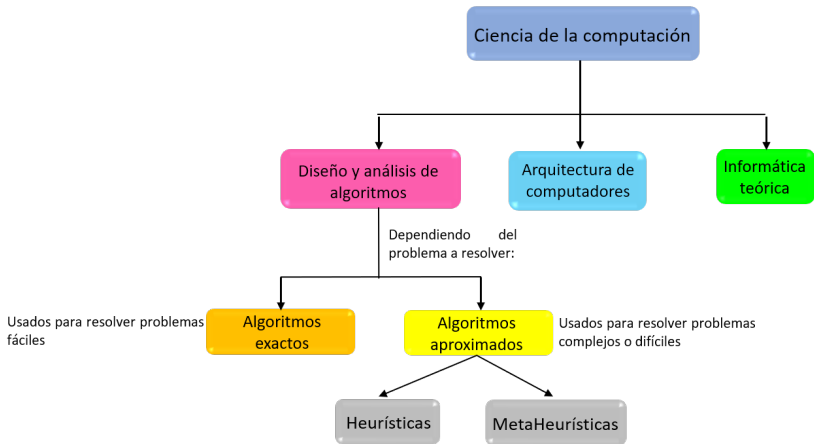
Estrategias exactas vs. aproximadas



1 Estrategias exactas vs. aproximadas

- Problema VRP para comparar las estrategias
- VRP: Modelo de opt. vs. Recocido simulado
 - Modelo PLEM
 - Recocido simulado
- VRP: Fuerza bruta vs. Recocido simulado
 - Seudocódigo fuerza bruta
- VRP: Alg. voraz vs. Recocido simulado
 - Seudocódigo voraz

Estrategias exactas vs. aproximadas



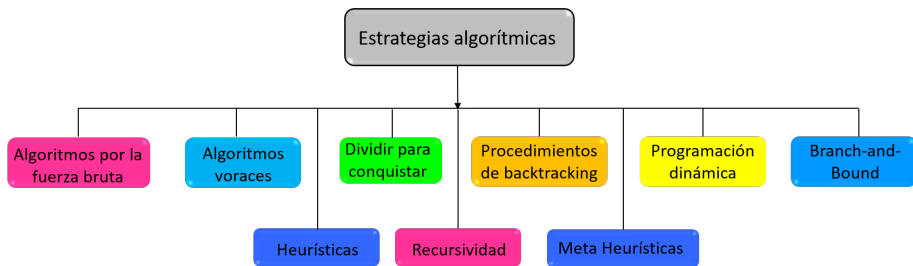


Tabla 1: Comparación general: Estrategia exacta vs. estrategia aproximada

Criterio	E. Exacta	E. Aproximada
Objetivo	Hallar solución óptima	Hallar soluciones aproximadas.
Solución	Garantiza la mejor solución.	No garantiza hallar la mejor solución, pero son eficientes.
Escalabilidad	Pueden ser impracticables para problemas grandes.	Mejor escalamiento para problemas grandes.
Escenarios	Problemas pequeños a medianos con soluciones óptimas.	Problemas grandes donde el óptimo es difícil de alcanzar.
Ejemplos	PL; PLEM; Branch-and-Bound; fuerza bruta.	"Alg. voraz"; Metaheurísticas(AG; ACO; SA; VNS; etc.).

Problema VRP para comparar las estrategias

PROBLEMAS DE VRP (VEHICLE ROUTING PROBLEM)



OBJETIVO:

Diseñar rutas para una flota de vehículos que partiendo y regresando a un depósito satisfagan la demanda de los clientes al mínimo costo (distancia, tiempo o costo), cumpliendo restricciones de capacidad y tiempo.

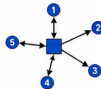
- Depósito
- Cliente
- Ruta vehículo

ESTRATEGIAS EXACTAS (Garantizan la solución óptima)



1. PROGRAMACIÓN ENTERA MIXTA (MIP)

- Formula el VRP como un modelo matemático de optimización con variables binarias y continuas.
- Se resuelve con algoritmos MIP (Branch & Bound, Branch & Cut, etc.).



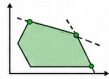
2. BRANCH & BOUND (B&B)

- Explora el espacio de soluciones mediante ramas (decisiones) y cotas (límites inferior y superior).
- Elimina ramas que no pueden mejorar la solución actual.



3. PLANOS DE CORTE (CUTTING PLANES)

- Comienza con una relajación del modelo MIP y añade cortes (restricciones válidas).
- Iterativamente mejora la relajación hasta obtener la solución óptima.



VENTAJAS:

Garantizan optimalidad y factibilidad.

DESVENTAJAS:

Tiempo de cómputo crece exponencialmente con el tamaño del problema.

ESTRATEGIAS APROXIMADAS

(Obtienen buenas soluciones en tiempos razonables)

HEURÍSTICAS

(Constructivas y de Mejora)

Constructivas



- Ahorros de Clarke & Wright
- Inserción más barata
- Vecino más cercano

De mejora local



- 2-opt
- Or-opt
- Relocate
- Exchange



VENTAJAS:

Simples, rápidas y fáciles de implementar.

DESVENTAJAS:

No garantizan optimalidad.

METAHEURÍSTICAS

(Mejoran iterativamente la solución)



Búsqueda Tabú (TS)

Explora vecindarios evitando movimientos recientes.



Recocido Simulado (SA)

Acepta peores soluciones con cierta probabilidad para escapar de óptimos locales.



Algoritmos Genéticos (GA)

Usan selección, cruce y mutación para evolucionar soluciones.



Colonia de Hormigas (ACO)

Construye rutas usando feromonas y refuerza buenas soluciones.



Búsqueda Local Iterada (ILS)

Perturba la solución y aplica búsqueda local repetidamente.

VRP: Modelo de opt. vs. Recocido simulado

- Estrategia exacta:

¹Simulated Annealing

VRP: Modelo de opt. vs. Recocido simulado

- Estrategia exacta:
 - ▶ Modelo de optimización.

¹Simulated Annealing

VRP: Modelo de opt. vs. Recocido simulado

- Estrategia exacta:
 - ▶ Modelo de optimización.
- Metaheurística:

¹Simulated Annealing

VRP: Modelo de opt. vs. Recocido simulado

- Estrategia exacta:
 - ▶ Modelo de optimización.
- Metaheurística:
 - ▶ Recocido simulado¹

¹Simulated Annealing

Modelo PLEM

- Conjuntos:

- ▶ N : Conjunto de nodos, donde el nodo 0 es el depósito.
- ▶ $C = N \setminus \{0\}$: Conjunto de clientes.
- ▶ K : Conjunto de vehículos.

Modelo PLEM

- Conjuntos:

- ▶ N : Conjunto de nodos, donde el nodo 0 es el depósito.
- ▶ $C = N \setminus \{0\}$: Conjunto de clientes.
- ▶ K : Conjunto de vehículos.

- Parámetros:

- ▶ d_{ij} : Distancia euclideana entre nodos i y j , para $i \neq j$.
- ▶ $n = |C|$: Número de clientes.
- ▶ c : Costo fijo por vehículo.

Modelo PLEM

- Conjuntos:

- ▶ N : Conjunto de nodos, donde el nodo 0 es el depósito.
- ▶ $C = N \setminus \{0\}$: Conjunto de clientes.
- ▶ K : Conjunto de vehículos.

- Parámetros:

- ▶ d_{ij} : Distancia euclideana entre nodos i y j , para $i \neq j$.
- ▶ $n = |C|$: Número de clientes.
- ▶ c : Costo fijo por vehículo.

- Variables de decisión:

$$x_{ijk} = \begin{cases} 1 & \text{si el vehículo } k \text{ viaja de } i \text{ a } j, \\ 0 & \text{en otro caso} \end{cases} \quad \forall i, j \in N, i \neq j, k \in K.$$

$$u_i \in [1, n] \quad \forall i \in C$$

- Función objetivo:

$$\min Z = \sum_{k \in K} \sum_{i, j \in N: i \neq j} d_{ij} \cdot x_{ijk} + c \cdot \sum_{k \in K} \sum_{j \in C} x_{0jk}$$

- Función objetivo:

$$\min Z = \sum_{k \in K} \sum_{i, j \in N: i \neq j} d_{ij} \cdot x_{ijk} + c \cdot \sum_{k \in K} \sum_{j \in C} x_{0jk}$$

- Restricciones:

$$\sum_{k \in K} \sum_{\substack{i \in N \\ i \neq j}} x_{ijk} = 1 \quad \forall j \in C \quad (\text{Cada cliente es visitado una vez}) \quad (1)$$

$$\sum_{k \in K} \sum_{\substack{j \in N \\ j \neq i}} x_{ijk} = 1 \quad \forall i \in C \quad (\text{Cliente es atendido una vez}) \quad (2)$$

$$\sum_{j \in C} x_{0jk} = \sum_{i \in C} x_{i0k} \quad \forall k \in K \quad (\text{Flujo balanceado en depósito}) \quad (3)$$

- **Restricciones:** (cont.)

$$\sum_{j \in C} x_{0jk} \geq 1 \quad \forall k \in K \quad (\text{vehículo atiende a un cliente}) \quad (4)$$

$$\sum_{\substack{i \in N \\ i \neq h}} x_{ihk} = \sum_{\substack{j \in N \\ j \neq h}} x_{hjk} \quad \forall h \in C, \forall k \in K \quad (\text{Balance de flujo en clientes}) \quad (5)$$

$$u_i - u_j + n \cdot x_{ijk} \leq n - 1 \quad \forall i, j \in C, i \neq j, \forall k \in K \quad (\text{Elimina subtours}) \quad (6)$$

- Restricciones: (cont.)

$$\sum_{j \in C} x_{0jk} \geq 1 \quad \forall k \in K \quad (\text{vehículo atiende a un cliente}) \quad (4)$$

$$\sum_{\substack{i \in N \\ i \neq h}} x_{ihk} = \sum_{\substack{j \in N \\ j \neq h}} x_{hjk} \quad \forall h \in C, \forall k \in K \quad (\text{Balance de flujo en clientes}) \quad (5)$$

$$u_i - u_j + n \cdot x_{ijk} \leq n - 1 \quad \forall i, j \in C, i \neq j, \forall k \in K \quad (\text{Elimina subtours}) \quad (6)$$

- Alcance de las variables:

$$x_{ijk} \in \{0, 1\} \quad \forall i, j \in N, i \neq j, k \in K$$

$$u_i \in [1, n] \quad \forall i \in C$$

- Ejemplo:

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- ▶ $K = 3$ vehículos

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- ▶ $K = 3$ vehículos

- ▶ $c = 20$ Costo fijo

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- ▶ $K = 3$ vehículos

- ▶ $c = 20$ Costo fijo

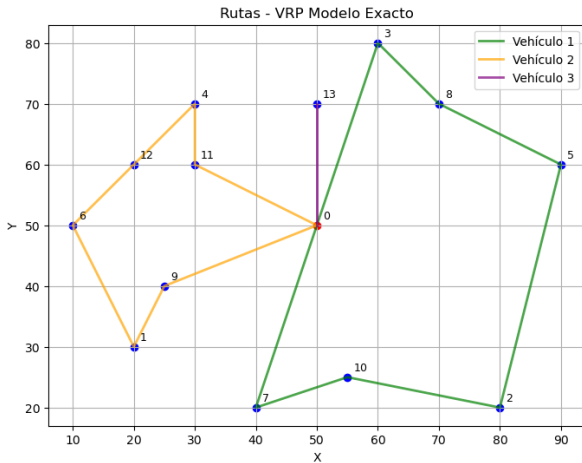
- ▶ Resultado

— Resultados VRP —

Estado de la solución: Optimal

Costo total: 403.39770544770795

● Ejemplo: (cont.)



Recocido simulado²

- Seudocódigo:

- 1: Definir coordenadas de nodos y lista de clientes
- 2: Calcular la distancia euclidiana entre los nodos i y j
- 3: Llamar a SimulatedAnnealing()
- 4: Obtener RutasFinales y CostoFinal
- 5: Imprimir resultados: Costo total y Rutas para cada vehículo
- 6:
- 7: **function** GENERAR SOLUCION INICIAL()
- 8: Mezclar aleatoriamente la lista de clientes
- 9: Inicializar K listas vacías para las rutas
- 10: Asignar los clientes a las rutas de manera circular
- 11: **for** cada ruta **do**
- 12: agregar el depósito al inicio y final
- 13: **end for**
- 14: Retornar las rutas
- 15: **end function**

²Simulated Annealing - SA

- Seudocódigo: (cont.)

```
1: function CALCULAR COSTO(rutas)
2:   Inicializar CostoTotal = 0
3:   for cada ruta en rutas do
4:     for cada par consecutivo de nodos,  $i; j$ , en la ruta do
5:       CostoTotal := CostoTotal + distancia(i,j)
6:     end for
7:   end for
8:   Retornar CostoTotal
9: end function
```

- Seudocódigo: (cont.)

- 1: **function** GENERAR VECINO(rutas)
- 2: Copiar las rutas, quitando los depósitos (primer y último nodo)
- 3: Crear lista de índices (Rutaldx, Clienteldx) para todos los clientes
- 4: Seleccionar aleatoriamente dos clientes distintos (de rutas posiblemente distintas)
- 5: **if** los clientes están en rutas diferentes **then**
- 6: Intercambiar los clientes
- 7: **end if**
- 8: **if** están en la misma ruta **then**
- 9: Reordenar aleatoriamente la ruta
- 10: **end if**
- 11: Volver a agregar los depósitos al inicio y final de cada ruta
- 12: Retornar las nuevas rutas
- 13: **end function**

● Pseudocódigo: (cont.)

```
1: function SIMULATEDANNEALING(MaxIter, TempIni, TempFin,  $\alpha$ )
2:   SolActual := Generar solución inicial( )           ▷ Sol.inicial
3:   CostoActual := CalcularCosto(SolActual)
4:   MejorSol := SolActual; MejorCosto := CostoActual   ▷ Inicializarlos
5:   Temperatura := TempIni                             ▷ Inicializar temperatura
6:   for Iter = 1: MaxIter do
7:     Generar vecino = GenerarVecino(SolActual)
8:     Calcular CostoVecino; Calcular  $\Delta$  = CostoVecino - CostoActual
9:     if  $\Delta < 0$  o random  $< e^{(-\Delta / Temperatura)}$  then
10:      SolActual := vecino                             ▷ Aceptar vecino
11:      CostoActual := CostoVecino
12:      if CostoVecino < MejorCosto then
13:        Actualizar MejorSol := vecino
14:        MejorCosto := CostoVecino
15:      end if
16:    end if
17:    temperatura = temperatura *  $\alpha$                  ▷ Reducir temperatura
18:    if Temperatura < TempFin then
19:      Terminar el ciclo
20:    end if
21:  end for
22:  Retornar MejorSol y MejorCosto
23: end function
```

- Ejemplo:

- Ejemplo:

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- ▶ $K = 3$ vehículos

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- ▶ $K = 3$ vehículos

- ▶ $c = 20$ Costo fijo

● Ejemplo:

▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

▶ $K = 3$ vehículos

▶ $c = 20$ Costo fijo

▶ Resultado

— Resultados Simulated Annealing —
Costo total: 416.92042597682786

● Ejemplo: (cont.)

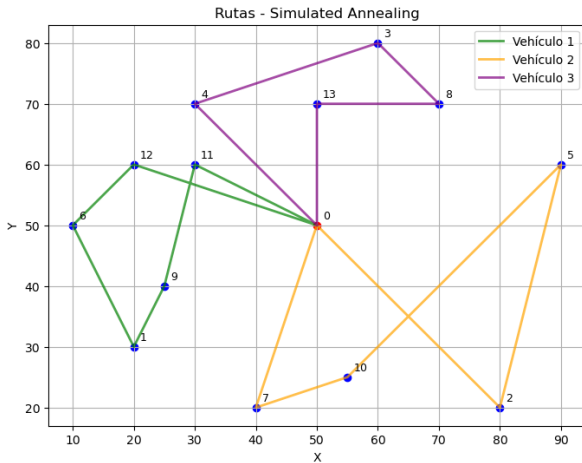


Tabla 2: Modelo PLEM vs. Simulated Annealing (SA)

Criterio	Modelo PLEM	Simulated Annealing (SA)
Tipo de solución	Exacta (óptima): 403	Aproximada (buena): 416
Librería	PuLP(solvers CBC, CPLEX, GLPK) + Python	Python puro
Estrategia	Programación Lineal Entera	Búsqueda local con enfriamiento simulado
Asignación de clientes	Cada cliente atendido una sola vez	Cada cliente asignado a una ruta sin repeticiones
Subrecorridos	Eliminados	Evitados debido a la generación y aceptación de vecinos
Num. vehículos	Fijo	Fijo
Costo final	Óptimo garantizado	Aproximado, mejora con más iteraciones
Tiempo de ejecución	Alto para problemas grandes	Bajo a moderado, depende de iteraciones

Criterio	Modelo PLEM (PuLP)	Simulated Annealing (SA)
Calidad de rutas	Mejor solución	Típicamente entre 2-10% del óptimo
Eficiencia en escalabilidad	Baja a moderada, crece exponencialmente cuando aumenta el número de nodos	Alta, puede manejar más nodos con buena calidad
Dependencia de parámetros	No requiere ajuste manual	Requiere ajustar temperatura inicial, final, tasa de enfriamiento, iteraciones
Visualización	Gráfica de rutas óptimas	Gráfica de rutas encontradas
Robustez	Puede quedarse atrapado o tardar mucho en casos complejos	Más robusta para problemas grandes, aunque no siempre consistente
Mejora de solución	Difícil, requiere reformulación o solver potente	Puede mejorar iterativamente con más tiempo o ajuste de los parámetros

VRP: Fuerza bruta vs. Recocido simulado³

- **Algoritmo de fuerza bruta:** Genera todas las permutaciones posibles de visitas a clientes, para tal efecto dividir en rutas para cada vehículo y calcular la distancia total para cada permutación. Finalmente, mantener la mejor solución encontrada.

³Simulated Annealing-SA

VRP: Fuerza bruta vs. Recocido simulado³

- **Algoritmo de fuerza bruta:** Genera todas las permutaciones posibles de visitas a clientes, para tal efecto dividir en rutas para cada vehículo y calcular la distancia total para cada permutación. Finalmente, mantener la mejor solución encontrada.
- **Observación:** Fuerza bruta soluciona el VRP en forma exhaustiva, garantizando encontrar la solución óptima. Sin embargo existe la posibilidad de ser ineficiente cuando la cantidad de clientes es grande, debido a la complejidad factorial de las permutaciones.

³Simulated Annealing-SA

Seudocódigo fuerza bruta

● Seudocódigo:

```
1: coords:= Lista de posiciones de los nodos
2:  $K$ := número de vehículos
3: nodos:= lista de coords
4: MejorSolucion:= nulo
5: MejorDistancia:=  $\infty$ 
6: function DISTANCIA(p1, p2)
7:   dx:= p1.x - p2.x
8:   dy:= p1.y - p2.y
9:   Retornar RaízCuadrada( $dx^2 + dy^2$ )
10: end function
```

- ▷ Coordenadas de nodos
- ▷ Incluye el depósito 0

● Seudocódigo: (cont.)

```
1: function CALCULARRUTA(ruta, coords)
2:   RutaTotal:= 0
3:   for i = 0 : Longitud(ruta) - 2 do
4:     NodoActual:= ruta[i]                                ▷ Trabajar con el nodo i
5:     SiguienteNodo:= ruta[i + 1]                          ▷ Trabajar con el nodo i + 1
6:     RutaTotal:= RutaTotal + distancia(coords[NodoActual],
      coords[SiguienteNodo])
7:   end for
8:   RutaTotal:= RutaTotal + distancia(coords[ruta[-1]], coords[0]) ▷ Agregar la
      distancia de regreso al depósito
9:   Retornar RutaTotal
10: end function
11: Clientes:= Nodos excluyendo el nodo 0
12: permutaciones:= GenerarPermutaciones(clientes)          ▷ Generar todas las
      permutaciones posibles de los clientes (excluyendo depósito)
```

● Seudocódigo: (cont.)

```
1: for cada permutacion en permutaciones do
2:   rutas:= lista vacía
3:   LongitudRuta:= Longitud(permutacion) / K      ▷ Dividir la permutación en
   rutas para cada vehículo
4:   índice:= 0
5:   while índice < Longitud(permutacion) do
6:     Ruta:= Sublista de permutacion desde índice hasta índice +
   LongitudRuta
7:     agregar ruta a rutas ; índice:= índice + LongitudRuta
8:   end while
9:   DistTotal:= 0                                ▷ Distancia total
10:  for Cada ruta en rutas do                    ▷ Calcular DistTotal para todas las rutas
11:    RutaCompleta:= [0] + ruta + [0]
12:    DistTotal:= DistTotal + CalcularRuta(RutaCompleta, coords)
13:  end for
14:  if DistTotal < MejorDistancia then          ▷ Verifica si solución actual es mejor
   que la anterior
15:    MejorSolucion:= rutas; MejorDistancia:= DistTotal
16:  end if
17: end for
18: Mostrar MejorSolucion y MejorDistancia
```

- Ejemplo:

- Ejemplo:

- ▶ N :

0: (50, 50),

1: (20, 30),

2: (80, 20),

3: (60, 80),

4: (30, 70),

5: (90, 60),

6: (10, 50),

7: (40, 20),

8: (70, 70),

9: (25, 40),

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),		

- ▶ $K = 3$ vehículos

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),		

- ▶ $K = 3$ vehículos

- ▶ Resultado

— Resultados Fuerza bruta —

Rutas: [(1, 7, 2), (3, 8, 5), (4, 6, 9)]

Distancia total mínima: 351.72137056533614

●



Tabla 3: Fuerza Bruta vs. Simulated Annealing (SA)

Criterio	Fuerza Bruta	Simulated Annealing
Solución	Óptima: 351	Aproximada: 416
Escalabilidad	Limitada para pequeñas instancias	Alta, manejable para instancias grandes
Implementación	Sencilla	Requiere más parámetros y diseño
Robustez	No robusto para grandes instancias	Alta, adapta soluciones con calidad
Tiempo de proceso	Lento para $n > 10$	Rápido para grandes instancias
Uso de memoria	Muy alto (almacena todas las permutaciones)	Bajo, solo la solución actual y algunas candidatas
Determinismo	Determinista (siempre el mismo resultado)	Estocástico (resultados pueden variar)
Proceso de mejora	No mejora progresiva (salto al óptimo)	Mejoramiento progresivo con las iteraciones

VRP: Alg. voraz vs. Recocido simulado⁴

- Estrategia voraz es una estrategia que siempre busca la mejor opción que tiene hasta ese momento.

⁴Simulated Annealing-SA

VRP: Alg. voraz vs. Recocido simulado⁴

- Estrategia voraz es una estrategia que siempre busca la mejor opción que tiene hasta ese momento.
- Aplicado al VRP:

⁴Simulated Annealing-SA

VRP: Alg. voraz vs. Recocido simulado⁴

- Estrategia voraz es una estrategia que siempre busca la mejor opción que tiene hasta ese momento.
- Aplicado al VRP:
 - ▶ Cada vehículo empieza su recorrido desde el depósito (0), y siempre elige el cliente que este más cercano a el y que aún no ha sido visitado.

⁴Simulated Annealing-SA

VRP: Alg. voraz vs. Recocido simulado⁴

- Estrategia voraz es una estrategia que siempre busca la mejor opción que tiene hasta ese momento.
- Aplicado al VRP:
 - ▶ Cada vehículo empieza su recorrido desde el depósito (0), y siempre elige el cliente que este más cercano a el y que aún no ha sido visitado.
 - ▶ La asignación de clientes a los vehículos es en forma secuencial.

⁴Simulated Annealing-SA

VRP: Alg. voraz vs. Recocido simulado⁴

- Estrategia voraz es una estrategia que siempre busca la mejor opción que tiene hasta ese momento.
- Aplicado al VRP:
 - ▶ Cada vehículo empieza su recorrido desde el depósito (0), y siempre elige el cliente que este más cercano a el y que aún no ha sido visitado.
 - ▶ La asignación de clientes a los vehículos es en forma secuencial.
 - ▶ Una vez asignados todos los clientes, cada recorrido se cierra regresando al depósito.

⁴Simulated Annealing-SA

Seudocódigo voraz

- Seudocódigo:

- 1: Coords ▷ Coordenadas de nodos incluido depósito
- 2: Definir número de vehículos K
- 3: Inicializar conjunto de clientes (excluyendo el depósito)
- 4: Crear lista de rutas: rutas[K] ▷ lista vacía para cada vehículo
- 5: DistRutas[K] := 0 ▷ Lista de distancias
- 6: PosActuales[K] := depósito (0) ▷ Lista de posiciones actuales

● Seudocódigo: (cont.)

```
1: while clientes no esté vacío do
2:   for cada vehículo  $k = 0 : K-1$  do
3:     if clientes está vacío then
4:       Salir del bucle para vehículos
5:     end if
6:     actual:= PosActuales[k]
7:     ClienteMasCercano:= NULL
8:     DistMinima:=  $\infty$  ▷ Distancia mínima
9:     for cada cliente en clientes do
10:      d:= distancia entre Coords[actual] y Coords[cliente]
11:      if  $d < \text{DistMinima}$  then
12:        DistMinima:= d
13:        ClienteMasCercano:= cliente
14:      end if
15:    end for
16:    Agregar ClienteMasCercano a rutas[k]
17:    Incrementar DistRutas[k] en DistMinima
18:    Actualizar PosActuales[k]:= ClienteMasCercano
19:    Remover ClienteMasCercano de clientes
20:  end for
21: end while
```

- Seudocódigo: (cont.)

- 1: **for** cada vehículo $k = 0 : K-1$ **do**
- 2: Calcular distancia de regreso al depósito
- 3: $dRegreso :=$ distancia entre $Coords[PosActuales[k]]$ y $Coords[0]$
- 4: Incrementar $DistRutas[k]$ en $dRegreso$
- 5: **end for**
- 6: Retornar rutas; distancias de cada vehículo y calcular distancia total de todos los vehículos

- Ejemplo:

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- ▶ $K = 3$ vehículos

- Ejemplo:

- ▶ N :

0: (50, 50),	1: (20, 30),	2: (80, 20),	3: (60, 80),
4: (30, 70),	5: (90, 60),	6: (10, 50),	7: (40, 20),
8: (70, 70),	9: (25, 40),	10: (55, 25),	11: (30, 60),
12: (20, 60),	13: (50, 70)		

- ▶ $K = 3$ vehículos

- ▶ Resultado

— Resultados VRP —

Estado de la solución: Optimal

Distancia total de todos los vehículos: 356.72

● Ejemplo: (cont.)

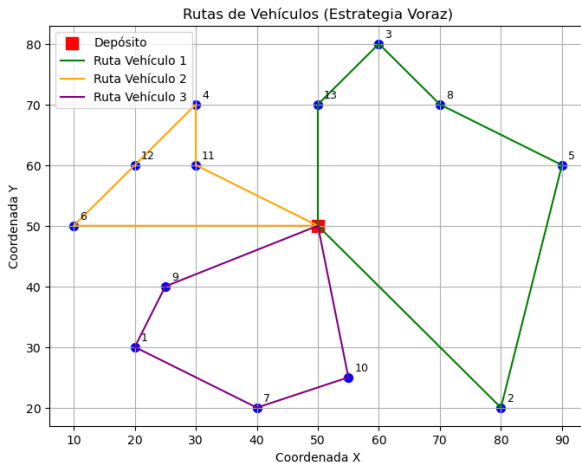


Tabla 4: Algoritmo voraz vs. Simulated Annealing (SA)

Criterio	Algoritmo Voraz	Simulated Annealing
Principio básico	Selección secuencial del cliente más cercano	Búsqueda iterativa con aceptación probabilística de soluciones para evitar óptimos locales
Naturaleza	Heurístico determinista	Metaheurístico estocástico
Exploración del espacio de soluciones	Explora sólo una solución local	Explora múltiples soluciones mediante perturbaciones controladas
Calidad de solución	Rápido pero propenso a óptimos locales; calidad variable	Generalmente encuentra soluciones de mejor calidad (cerca al óptimo global)
Tiempo de proceso	Rápido	Lento; depende de parámetros de enfriamiento
Facilidad de implementación	Sencillo de implementar	Requiere ajuste de parámetros como temperatura inicial, tasa de enfriamiento y Num. de iter.
Recomendación	Útil para soluciones rápidas y aproximadas	Recomendado para soluciones más robustas y optimizadas